From Connected Assets to Connected Intelligence

CONNECT → PREDICT → REASON → PRODUCTIZE → LEARN — and what it takes to sell the result

Kunjar Bhaduri · August 2026

> ***Scope.*** *This paper describes a reference implementation built on synthetic data and publicly documented patterns. It contains no proprietary architecture, data, or information from any operator or vendor. Every claim about the implementation is verifiable by running it; the figures are bound to a commit in* `RELEASE_EVIDENCE.md`*. Where this paper describes what the implementation demonstrates, it says so plainly; where it describes what the architecture merely suggests or what remains to be proven in the field, it is labeled* ***hypothesized*** *or* ***not demonstrated****, not left for the reader to infer.*

---

1. Executive summary

Connected-asset platforms are good at collecting telemetry and increasingly good at predicting what a sensor reading means. Neither of those is the hard problem anymore. The hard problem is turning a prediction into something an enterprise can hand to a customer, an operator, or a regulator — and still defend, on the specifics, eighteen months later.

This paper describes a reference implementation of a **portable Intelligence Product Contract**: a decision record that carries a prediction claim alongside the business consequence it threatens, the evidence available at the moment the decision was made, whether the organization was entitled to deliver it to that recipient, what was actually delivered, and what happened afterward. The implementation runs on synthetic data — one forklift, one loading window, one customer commitment — and demonstrates the full `CONNECT → PREDICT → REASON → PRODUCTIZE → LEARN` loop end to end, with **trust** as the property, threaded through all five stages, that this paper argues is the precondition for that loop to be usable at enterprise scale rather than staying a demo — an architectural claim, not a report that it has been run at that scale.

The central argument: a prediction is not a product. A product is a claim with evidence, consequence, authority, and a lifecycle attached — and most of the difficulty in "AI-powered" connected-asset intelligence lives in the part after the model produces a number, not the part before it.

2. The problem: connected data is plentiful, actionable trustworthy intelligence is not

Every connected asset — a forklift, a compressor, a vehicle, a piece of yard equipment — now produces a stream of telemetry. Storing it, joining it, and running a model over it have all gotten dramatically cheaper over the last decade. None of that is the bottleneck anymore.

The bottleneck is what happens after the model returns a number. A risk score of 0.49 is not a decision. It is not defensible in a dispute. It carries no statement of what business outcome it threatens, whether the organization had the right to compute or deliver it, what was actually known when it was produced, or what happened after someone acted — or didn't — on it. An operator sitting on years of telemetry and a working model can still have nothing they can respectably sell, because none of that surrounding structure exists.

This paper is about that surrounding structure — what it has to contain, and what a reference implementation of it actually looks like when it runs.

3. Why prediction is not the product

Forklift F-17 works an indoor loading yard. Not a truck on a highway — an on-site asset, the class growing fastest and worst served by platforms built for on-road telematics. Indoors there is no satellite positioning, the feed reconnects in bursts, and the machine moves between bays hourly.

F-17 is running hot. Over five days its battery pack reports 41.0, 45.6, 51.4, 58.4, 66.6 °C — not a drift, an acceleration.

A sensor threshold says: *battery temperature high.* A model says: *degradation risk 0.49.* Neither is a product. Both are things a maintenance team already has too many of.

Now add four facts that live in systems entirely separate from the telemetry. F-17 carries battery-pack-1. F-17 is assigned to loading zone 3. Zone 3 loads shipment 22 tomorrow morning. Shipment 22 fulfils a next-day delivery commitment for customer A.

Now the reading says something a customer would pay for:

> *This asset is likely to be unavailable during tomorrow's loading window, which puts a next-day delivery commitment at risk. Inspect it before the next shift.*

That sentence is the product. Everything in this paper is about what it takes to produce it, deliver it, and still be able to defend it eighteen months later.

When a prediction is delivered to a customer, it stops being an analytics output and becomes a commercial claim. It carries an implied promise. It implies the provider had the right to use the data behind it. And it creates an obligation — sometimes months later, sometimes under adversarial conditions — to explain how it was produced. An organization that has not built for that third obligation discovers it at the worst possible moment.

4. The five-stage operating loop

```
CONNECT    enough point-in-time context to know what physical thing this is,
           where it was, what it was doing, and what depended on it
PREDICT    a bounded claim about the next 72 hours
REASON     which business entities and commitments that claim affects
PRODUCTIZE whether the resulting intelligence may be delivered at all
LEARN      what the recipient did, what happened, what improves the next one
```

The first three make intelligence possible. The fourth makes it sellable. The fifth makes it compound. **Trust** is not a sixth stage in this sequence — it is a cross-cutting property, covered in §10, that this paper argues is the precondition for running the loop above at enterprise scale rather than staying a research notebook. That argument is architectural, not a claim that this reference implementation has been run at enterprise scale — it has not; see §19. Trust arrives after the five stages in this paper's structure for the same reason it is not first in the diagram.

The forklift example is deliberately an on-site asset. A platform that understands the yard *and* the road can see the asset, the bay, the window, and the shipment that depends on all three; a single-context platform cannot. That is where the compounding starts.

5. CONNECT

CONNECT answers one question: what physical thing are we talking about, where was it, what was it doing, and what depended on it — as of a specific, named moment, not as of now.

Connected assets do not report in real time. A device buffers while out of coverage — in a yard, indoors, or switched off — then delivers a backlog on reconnect. Delivery is at-least-once, sources disagree, and clocks drift. In the reference implementation, three independent sources feed one asset: an on-network telemetry stream, a channel-partner feed on its own identifier scheme roughly two hours behind, and a position-denied yard sensor that delivers its backlog in bursts. Every reading from every source is recorded with both its **valid time** (when it was true of the world) and its **transaction time** (when the system learned it) — the mechanism §12 covers in detail.

CONNECT's output is not a merged table. It is a point-in-time view a later stage can query honestly: *what was known as of this exact moment*, computed rather than remembered.

6. PREDICT

PREDICT turns the point-in-time view into a bounded claim: a score, a horizon, a declared applicability envelope, and a record of exactly which inputs — by id and content digest — produced it.

The implementation's bundled predictor is a monotonic degradation fit, deliberately simple enough to verify by hand: it declares the metric and horizon it is valid for and refuses outside that envelope, is deterministic so a stored claim can be re-checked years later, and states its output plainly as a relative risk ordering, not a calibrated probability of real-world failure. **Substituting a more**

sophisticated model changes nothing downstream — the contract described in §11 is independent of the model's sophistication. That independence is a design property of the contract, not a claim about how good this particular predictor is.

A prediction claim on its own is still not a product. It is the input to REASON.

---

7. REASON

A prediction about a battery is not yet a statement about the business. REASON walks from the predicted component outward — component, zone, loading window, shipment, customer commitment — and answers which entities and which commitments a prediction actually threatens, using only relationships that were themselves known by the decision time.

Most predictive-maintenance stacks stop one layer short of this. Without it, the system can say a battery is degrading. With it, the system can say which customer promise is at risk, by when, and why. §14 covers the mechanics — and the boundary of what is genuinely new here versus already well-served by existing tooling — in full.

---

8. PRODUCTIZE

A prediction and a derived consequence are still not a product until a system decides, for a named recipient, whether the resulting claim may actually be delivered — and carries the means to prove that decision was sound, later, outside the system that made it.

Be precise about what "product" means in this section, because the word carries a narrower claim here than a product leader might expect from the heading. It means a decision record that is defensible — release-authorized, evidenced, rights-checked, and reconstructable — not a packaged, priced, go-to-market-ready commercial offering. Pricing, packaging, and commercial adoption are explicitly not demonstrated here; see §19 and §20. What PRODUCTIZE argues is a precondition for selling this kind of intelligence, not that this reference implementation has been sold.

PRODUCTIZE is where this paper's central technical claim lives, and it is large enough to need its own sections: the contract the delivered unit carries (§11), how temporal honesty is enforced so a decision is judged on what was knowable rather than what is now known (§12), how evidence is captured so a claim can be reconstructed rather than merely asserted (§13), how a right to deliver is recorded rather than assumed (§15), and how release, refusal, and abstention actually work as typed outcomes (§16).

---

9. LEARN

LEARN captures what the recipient did with a delivered product and what was subsequently observed, joined to the specific recommendation that preceded it by id — not merely logged alongside it.

In the reference scenario: the product was delivered, acknowledged, accepted, the recommended inspection was completed, and a degraded cell was found and swapped before the loading window opened. That sequence is captured as a typed disposition trail plus a typed outcome observation, joined to the action's id.

What this stage does **not** do matters more than what it does. `attributed_effect` — a field on the exported envelope — has exactly one legal value, `NOT_ESTABLISHED`, and a test in the suite fails if a second value is ever added. Nothing was randomized in the scenario and the recipient chose whether to act; "acted, and no failure occurred" is not evidence the recommendation caused that outcome. LEARN demonstrates outcome **capture** and the join structure a learning loop needs. It does not demonstrate that a learning loop has been run, or that retraining on captured outcomes improves anything — see §19.

---

10. TRUST as the cross-cutting property

Trust is not a feature bolted onto the loop above. It is the set of properties, present in each stage, that determine whether an enterprise can actually rely on the loop's output without re-deriving it themselves. Naming it is cheap; the point of this section is the mechanics, checkable in the code and the schema rather than asserted in prose:

- **Temporal integrity (CONNECT, PRODUCTIZE).** Every observation and every derived relationship carries valid time and transaction time; a decision is reconstructed against the view that existed at the decision, never against hindsight. Enforced by `cai.bitemporal` and checked by the schema constraint `knowledge_cutoff <= model_decision_time <= release_decision_time`. Detailed in §12.
- **Evidence integrity (PREDICT, PRODUCTIZE).** A claim carries the exact observations and their content digests that produced it; a prediction can be replayed from the recorded inputs and must reproduce the same score, the same fingerprint. Detailed in §13.
- **Identity (CONNECT, PRODUCTIZE).** Identity resolution confidence is carried as an explicit, bounded field on every claim rather than assumed to be perfect — the implementation demonstrates the *consequence* of identity uncertainty on a release decision, not identity resolution itself.
- **Rights and authority (PRODUCTIZE).** Entitlement is a typed field checked at release time, not a policy document consulted after the fact. Detailed in §15.
- **Reproducibility.** Every figure in this paper and in `AUDIT.md` was produced by a script in this repository, re-runnable by a reader against this exact commit. The test suite, the scenario, and the schema/code equality check are additionally re-checked in CI on every push. The mutation and guard-sweep figures are not CI-gated — deliberately, because that measurement is too expensive to pay on every push — and are reproducible on demand rather than continuously enforced; see `AUDIT.md` for exactly which figures carry which guarantee.
- **Explicit refusal.** `ABSTAINED` is a typed outcome with a mandatory, specific reason — never a silent no-op indistinguishable from a crash six months later.

- **Typed state, not strings.** The release path accepts no free-text business context, no free-text economic consequence — only a candidate whose consequence was derived by a graph walk the release path can itself inspect.
- **Import/release symmetry.** Reading a delivered product back re-runs the same evidence assessment that produced it and refuses on disagreement — an importer is not asked to trust the wire copy's derived fields.
- **Reconstruction.** A product exported today can be re-imported, replayed, and its score reproduced outside the process that generated it originally, with no access to the original telemetry store.
- **Lifecycle integrity.** An abstained product carries no lifecycle events, no outcome, and no status events by schema construction — a claim that was withheld could not have been acknowledged, acted on, or observed.
- **Rendered-artifact binding.** What a recipient was actually shown is captured as a digest-verified rendered artifact, distinct from — and checked against — the typed fields that produced it.
- **Explicit limitations.** Every boundary this implementation has is stated in `README.md`, this paper, and `AUDIT.md`, not left for a reader to discover by probing.

Trust, in other words, is not a claim this paper makes about itself. It is the sum of the mechanisms above, each independently testable, and `AUDIT.md` describes how each one is actually verified.

---

11. The portable Intelligence Product Contract

Treat a delivered piece of intelligence as a first-class product unit with an explicit contract, rather than as a row in an output table. In plain terms, the unit carries:

```
subject                what asset, component, site, or workflow
claim                  what is predicted or inferred
prediction_horizon     by when
business_context       why it matters operationally
economic_consequence   what revenue, cost, or commitment is affected
recommended_action     what the recipient should do
confidence             how certain
abstention_reason      why no claim was issued
evidence_snapshot      what was known at delivery time
model_version          which predictor
semantic_rule_version  which business rules
release_policy_version which thresholds authorised delivery
customer_entitlement   whether delivery was permitted at all
recipient              who received it
delivery_time          when
disposition            what they did
observed_outcome       what actually happened
expiry                 when it should no longer be relied upon
```

The actual wire contract (`schema/envelope-1.json`, a normative Draft 2020-12 JSON Schema, generated from and checked against the code on every push) is considerably stricter than the field list above suggests: its `x-semantic-validations` block states 35 explicit cross-field invariants — a

claim's horizon must equal its prediction window's duration rather than being free to disagree with it; a business consequence's derivation must terminate at the consequence's own subject and use no fact recorded after the knowledge cutoff; an abstained product must carry zero lifecycle events; a rendered delivery's digest must verify against the exact fields that produced it — rather than merely typing the fields above. The field list is the product a business reader should recognize; the schema is what a wire implementation actually has to satisfy.

Precision has a boundary worth stating for a reader planning to build against this contract in another language. The 35 invariants above are structural and semantic rules a document either satisfies or does not, and the reference importer enforces every one of them by reconstruction — that part is fully specified, and a reimplementation in another language can validate structure and every invariant from the schema alone.

What the schema does not state — the exact digest algorithm behind each of the eight 64-character hex fields, the full rule set behind the `RFC8785-subset/1` canonicalization identifier, and the relationship-continuation rules a REASON derivation must satisfy — is now specified precisely, transcribed from the running implementation rather than from desired standards language, in `docs/PORTABILITY_CONTRACT.md`. The short version: digests are SHA-256 throughout, but computed by four different digest-input constructions depending on the field — two fields hash canonical JSON, several hash an ad-hoc delimiter-joined string that depends on CPython's own float formatting, one hashes this implementation's own source file bytes, and one hashes an order-independent aggregate of sorted child digests (themselves computed by the delimiter-joined construction). A reader in another language can validate every field's *shape* and every cross-field invariant from the schema alone; reproducing certain digest fields byte-for-byte additionally requires replicating two CPython-specific numeric formattings, which `docs/PORTABILITY_CONTRACT.md` names exactly. That is a real, now-precisely-bounded limit on cross-language interoperability, not a gap left for a reader to discover by reading Python source.

Two properties matter more than the field list itself.

The context is derived, not asserted. `business_context` is rendered from a graph walk — component, zone, loading window, shipment, commitment — and the release path refuses a product whose context was not derived. A caller cannot supply the sentence. This is the difference between the two systems it is easy to build by accident:

```
disparate data -> prediction -> derived context -> intelligence product     (the
claim)
supplied intelligence -> governance -> reconstruction                       (the
trap)
```

The second demos beautifully and proves nothing. An engineer finds it by reading the imports.

A decision is judged on what was knowable when it was made, covered next.

12. Temporal integrity and avoiding hindsight

"What did the system know on day 12" and "what do we now believe was true on day 12" are different questions, and a record with one timestamp cannot tell them apart. Every observation carries valid time and transaction time. In the reference scenario, 9 observations were available at the decision, 13 are believed in hindsight, and 4 were learned only afterward. The claim is judged on 9.

This applies to relationships, not only readings — and that generalization is easy to miss. A zone reassignment entered into the system after the decision cannot be used to justify it, however true it later proved. Getting this right for telemetry and wrong for business relationships breaks decision-time honesty at exactly the layer that turns a prediction into intelligence.

The schema encodes the ordering directly: `knowledge_cutoff <= model_decision_time`, and `model_decision_time <= release_decision_time` for every determination — delivered or abstained. These are different domain events. Model decision time bounds the evidence a claim may use; release decision time selects which policy version governs it; delivery time is when a recipient actually received the product. Equality between them is not required; the ordering is.

13. Evidence and reconstruction

A claim that cannot be reconstructed from a record is not evidence of anything except that a number was once computed. The reference implementation's evidence bundle carries the exact observation ids and a content digest of each one; a prediction can be replayed by re-running the recorded predictor over the recorded inputs, and the replay must reproduce the same score, evidence strength, and input fingerprint — checked on release by construction, not merely available for a curious reader to attempt.

Two consequences follow. First, evidence sufficiency is a **pre-delivery control**, not a post-hoc audit: no claim is released unless the evidence needed to defend it later can already be assembled. Evidence gathered after a dispute begins is a reconstruction of a reconstruction. Second, the digest is tamper-**evident**, not tamper-proof — a party who controls both the record and the underlying readings can alter them together and recompute a consistent digest. That is a stated limitation, not an oversight; see §19.

14. Context and consequence reasoning

It is worth being precise about what does *not* need building, because a great deal of governance discourse consists of rediscovering solved problems.

Concern	Already solved by
How was this dataset produced?	OpenLineage, Marquez, DataHub, W3C PROV
Which model version, trained on what?	MLflow and equivalent registries
Are features point-in-time correct?	Feature stores
Does this data meet declared expectations?	Great Expectations, Soda
Which records refer to the same entity?	Mature entity-resolution products
Is this output drifting?	Model-monitoring platforms

Anyone proposing to rebuild these is producing a weaker copy of tools that already exist. Distinguishing certain answers from merely possible ones is likewise not new — certain-answer semantics over incomplete databases dates to Imieliński and Lipski (1984), and provenance semirings to Green, Karvounarakis, and Tannen (2007).

So the interesting question is not "how do we track lineage." It is what remains untracked when all of the above works perfectly.

Consider the claim about F-17 delivered to customer A, and the questions that arrive afterward: what exactly was promised, and by when; what was the recommended action; did the provider have the right to use that data for that purpose; was it entitled to deliver that insight to that recipient; what evidence existed at delivery time — not what exists now; which model and which business rules produced it, under which release policy; who received it, and what did they do; what actually happened; and may that outcome be reused to improve the product for other customers?

A precisely instrumented data platform can leave every one of these unanswerable, because they are *product*-scoped and commercially oriented, while lineage is *dataset*-scoped and pipeline-oriented.

A note on precision here, because the easy version of this argument is wrong. It is tempting to say lineage tools "cannot model a physical asset." That is not true — W3C PROV models arbitrary entities, activities, and agents, and OpenLineage's core model is dataset/job/run by default rather than by limitation. Capability is not the issue. The defensible claim is narrower:

> *These concerns are not normally packaged together as the portable commercial unit being delivered to the customer.*

There is a further constraint that is easy to miss. The answers must survive **outside the platform that produced them**. Modern platforms export data well — open table formats, standard interfaces. They export *decisions* poorly: business logic, permissions, and workflow definitions generally do not travel. A decision record that exists only inside the system that made it is not much of a record — which is exactly what §10's reconstruction property and the portable envelope in `cai.envelope` are for.

15. Rights and authority

Cross-customer intelligence — benchmarking, peer comparison, network effects — is commercially attractive and is a **rights question before a technical one.** An operator frequently holds data it has no contractual right to monetize across customers.

Encoding entitlement as a field forces the question at the moment of delivery rather than during a contract dispute. In the reference implementation, a claim with complete evidence, high confidence, and resolved identity is refused — every technical signal green — because the operator had no right to deliver it to that specific recipient. The same operator, delivering the same claim, to the recipient it *is* entitled to serve, delivers successfully. Entitlement is scoped to recipient, purpose, tenant, subject, and the specific source systems the evidence actually drew on — a clearance obtained for one recipient does not authorize another, and one obtained for a narrower purpose does not cover a broader one.

To be exact about the boundary: this **records** a rights determination made by counsel or a policy engine. It does not interpret contracts or determine legal rights. Forcing the answer to be written down at the moment of delivery is the contribution, and it is a narrow one. See `SECURITY.md` for the same boundary stated plainly for a security reader.

16. Release, refusal, abstention, lifecycle, and outcome

In practice, a delivery decision reduces to six independent dimensions, held separately rather than collapsed into one verdict: **support** (does the evidence back the claim), **completeness** (is anything missing, and named if so), **integrity** (can the record itself be trusted), **temporal eligibility** (was the evidence available as of the decision), **identity** (is the subject resolved without ambiguity or conflict), and **entitlement** (may this be delivered, to this recipient, for this purpose). All six must be at their permitting value for release; any one blocking value produces a named, specific refusal reason rather than an undifferentiated "no."

That separation matters operationally, not just architecturally: a rights refusal routes to commercial, a broken evidence chain routes to engineering, a late arrival is nobody's fault and simply bounds what the decision could have known. Collapsing all six into a single provable/not-provable verdict — which an earlier specification of this system did — tells a reader something is wrong and nothing about what. `AUDIT.md` describes how this six-dimension algebra, all 432 enumerated states, and the one combination that releases, are verified by enumeration and by mutation rather than by inspection.

A withheld claim is not a failure state to be minimized to zero — abstention with a named reason is a valid, intended product outcome. `ABSTAINED` without a stated reason is a defect that raises an error in the implementation, because a silent abstention is indistinguishable from a crash in a log six months later. This paper's operating hypothesis, not a measured result, is that abstention rate and abstention *reasons* would become among the most useful operational signals a governed intelligence product could generate in production: each reason names a specific capability gap, so a team could in principle rank gaps by how many claims each one blocks. Whether that ranking actually predicts where investment pays off has not been tested against real operational data — see §19 and §20.

Once delivered, a product's lifecycle is a typed trail — delivered, acknowledged, accepted or overridden or ignored, action completed, outcome observed — and, separately, a typed outcome observation joined to the specific action it followed. The schema enforces that an abstained product carries none of this: a candidate that was withheld was never delivered, acknowledged, acted on, observed, retracted, or superseded. See §9 for what the outcome record does and does not claim.

---

17. The runnable synthetic scenario

`scenarios/incident_reconstruction.py` runs the entire argument of this paper as executable code, on synthetic data, in well under a second. It builds an observation log from three sources with different identifier schemes and arrival patterns, and a context graph carrying the component-to-commitment relationships described in §7 and §14. It then:

1. Computes, rather than asserts, what was knowable at the decision versus in hindsight (§5, §12).
2. Produces a bounded prediction claim from the observations available at that moment (§6).
3. Derives the business consequence by walking the context graph, and forms a candidate from the claim, the consequence, and a recommended action (§7).
4. Runs the release path four times: once where every dimension permits delivery, once where the recipient lacks entitlement, once where the evidence has been tampered with, and once where the prediction is correct but no business consequence is reachable at all — an alert, not a product (§16).
5. Advances a lifecycle trail through delivery, acknowledgment, acceptance, and a completed inspection, and records the observed outcome — a degraded cell found and replaced before the window opened (§9).
6. Exports the entire bundle as a canonical, digest-checkable envelope, re-imports it, and replays the prediction from the recorded inputs alone, reproducing the original score with a matching fingerprint and no access to the original telemetry store (§10, §13).

Every value printed by the scenario is computed by the modules described above at runtime — nothing in the printed narrative is a literal string standing in for a real computation. `verify_package.py` checks this specifically: it asserts, against the scenario's own source, that it invokes the predictor, the context graph, the lifecycle, and the release path by name, and that the claim it forms is built from the predictor's output rather than assembled by hand.

---

18. What the reference implementation demonstrates

By executable code, checked in CI on an exact commit and reproducible by any reader who clones the repository:

- An end-to-end CONNECT → PREDICT → REASON → PRODUCTIZE → LEARN chain in which each stage's output is computed, not typed in by hand.
- A six-dimension evidence algebra, 432 enumerated states, exactly one releasing, that can refuse a technically complete claim on entitlement alone.

- Bitemporal reconstruction that produces different, quantified answers for "known at the decision" versus "known in hindsight."
- A portable envelope that exports, re-imports, and replays a recorded prediction outside the process that produced it, reproducing the same score from a matching fingerprint.
- Typed abstention, with a mandatory reason, as a first-class product outcome.
- Outcome capture joined to a specific prior action by id.
- A wire schema that is provably identical, on every push, to the schema the code actually enforces.
- 1335 tests, two independent mutation techniques, and a guard sweep, each bounded and described in `AUDIT.md` — none summarized here as a single quality number, because that is precisely the kind of vanity metric this paper argues against.

---

19. What it does not demonstrate

The following are architecturally plausible and, in places, the entire reason the contract is shaped the way it is — but none of them has been shown by this implementation, and none should be read as a claim this paper makes:

Not demonstrated: production predictive accuracy or calibration; causal operational or financial benefit; real retraining performance or automatic model promotion; production-scale throughput or reliability; production telemetry from any real fleet, vehicle, or asset base, or integration with any named commercial vendor or platform; a legal determination of data rights; customer willingness to pay; commercial product-market fit; cross-customer benchmarking; that learning has occurred merely because outcomes are captured; proven reduction in downtime; proven improvement in safety; proven cost savings; proven revenue uplift.

Stated plainly, because a reader who finds one of these gaps should see it was already named:

- All data is synthetic. No claim is made that any result generalizes to production telemetry.
- Identity confidence is an **input**, not computed — the implementation demonstrates the *consequence* of identity uncertainty on a release decision, not its resolution.
- No positioning semantics for indoor or position-denied assets; only the arrival pattern of such sources is modeled.
- The observation log is append-only in-process, not cryptographically chained.
- A correction must explicitly name the reading it supersedes; an unannounced correction arriving under a new id is indistinguishable from a second reading of the same event.
- The evidence digest is tamper-**evident**, not tamper-proof — see §13.
- Release authority is a single global confidence floor, not scaled to the economic consequence of the claim.
- Entitlement records a determination made elsewhere; it does not interpret contracts.
- No watermarking or windowing policy — those belong to the ingesting engine.
- Only a `DELIVERED` product carries a rendered card. `abstention_reason` is a typed field on the record, consumed by audit and by whatever downstream system reads it — the reference

implementation does not itself render or send an abstention notice to the person who would have received the claim.
- The rendered card shows numeric scores (risk, identity confidence) alongside plain-language text, without a stated scale or severity anchor next to the numbers — reading urgency from the card currently presumes the reader already knows what the range means.

---

20. Potential applications and hypotheses

Offered explicitly as hypotheses, because the rest of this paper is verifiable and this section is not:

Intelligence becomes a product line, and therefore a liability surface. As predictive insight moves from an included feature to a paid product, it plausibly acquires the properties of a commercial commitment — expectations of accuracy, recourse when wrong, discovery exposure when consequential. This has not been tested against a real dispute.

Entitlement recorded early plausibly becomes the constraint that unlocks network effects, rather than blocking them. The most commercially attractive products in this space are cross-customer, and every one runs into data rights. An operator that tracked entitlement per claim from month one has an answerable question rather than a dispute that takes a quarter — this is an argument about incentive structure, not a measured result.

The closed loop, not raw data volume, is hypothesized to be what compounds into an advantage. If volume alone were decisive, the largest collector would already have won. What this paper's architecture is betting on is the loop — predict, act, observe, calibrate — and the belief that most organizations can capture the first two links today and not the last two. Whether closing the loop actually produces a measurable advantage is exactly the kind of claim that requires field data this reference implementation does not have.

The pattern may generalize beyond forklifts and beyond on-site industrial assets — to any connected system where a prediction has to become a decision another party acts on: fleet and transportation assets, industrial equipment more broadly, connected medical devices, infrastructure monitoring. Nothing in the contract in §11 is specific to batteries or forklifts. Whether the ontology and consequence types in §14 generalize without a rewrite is itself the first thing §21's roadmap tries to establish.

---

21. Path from reference implementation to real-world validation

A roadmap that names only destinations is a picture with dates attached. What follows is sequenced by what each stage *proves*, because the order is the argument, and it applies to any organization considering whether this pattern is worth adopting — not a commitment this project is making on a timeline.

Phase A — one product, shadow mode, fully instrumented. Ship a single narrow intelligence product on one asset class, delivered to nobody. Every claim runs the full path and is recorded; none reaches a customer. Instrument the four things that later become impossible to backfill: identity

resolution, the prediction and its inputs, the action taken, and the observed outcome. *Proves:* whether the abstention rate stabilizes and the abstention reasons sort into a ranked list of capability gaps — found at zero customer risk, because nothing was ever promised to anyone.

Phase B — cross-context reasoning, and a real measurement. Add the reasoning layer across business contexts and launch a small number of intelligence products around the same asset class. *Proves:* whether predictions measurably improve operations — not whether model accuracy improved, which is a more comfortable and different question.

Phase C — generalize the contract, not the code. Extend the product contract and semantic model to a second on-site asset class, and begin selective customer-facing delivery where Phase B's evidence supports it. *Proves:* whether the contract generalizes without a rewrite. If it needs one, the contract encoded one asset class's assumptions and §20's generalization hypothesis was premature.

Phase D — network effects, only if earned. Cross-customer learning, benchmarking, higher assurance tiers, and additional asset classes — and only where data rights and outcome evidence support it. This phase is last on purpose: it is the most commercially attractive category and the one most likely to be built first and unwound later. Recording entitlement per claim from day one is what turns this from a dispute into a question answerable in an afternoon.

Each phase's *proves* clause is a falsifiable claim about that phase, not an assumption carried in from this paper.

---

22. Conclusion

Everything above except one sentence is conventional engineering, deliberately so.

> ***The distinctive unit is a decision-time Intelligence Product Contract in which identity uncertainty and data rights directly govern whether a connected-asset prediction may be delivered.***
>
> *This composes established temporal-data, provenance, prediction, and policy concepts into one portable, versioned record. Decision-scoped provenance, bitemporal reconstruction, and release policy are all prior art. The contribution is the composition and the product boundary, not a new primitive and not new mathematics.*

An overstated claim in this territory is easy to puncture and deserves to be. What this paper asks a reader to accept is narrower and, we think, more useful: that the gap between a prediction and a product an enterprise can actually stand behind is real, that it is mostly not a modeling problem, and that a reference implementation exists that anyone can run, read, and try to break.

References

1. W3C, *PROV-DM: The PROV Data Model* — https://www.w3.org/TR/prov-dm/
2. OpenLineage, *Object Model* — https://openlineage.io/docs/spec/object-model

3. T. Imieliński and W. Lipski, "Incomplete Information in Relational Databases," *Journal of the ACM* 31(4), 1984.
4. T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance Semirings," *PODS*, 2007.
5. I. P. Fellegi and A. B. Sunter, "A Theory for Record Linkage," *JASA* 64(328), 1969.
6. Open Source Initiative, *MIT License* — https://opensource.org/license/mit

Reference implementation: `connected-asset-intelligence`*. Synthetic data throughout. MIT licensed. The typed domain imports only the standard library; the wire boundary depends on one declared third-party package,* `jsonschema`*, for schema validation; the test suite additionally requires pytest.*